

VIII. Documentation

A. Introduction

Bienvenue dans la documentation du programme final du projet « Des points et des lignes » de 2026 . Ce programme a pour but de traduire des croquis en maillage 2D pour le logiciel Gmsh.

Ce programme a été rédigé par **Luc Delépine, Jules Claus, Paul Correia, Stefan Stajkovski, Joao Paulino et Maxence Guerguil.**

Chaque personne a rédigé la section concernant ses programmes. Il y aura donc plusieurs styles de rédaction différents.

Nous avons implémenté deux méthodes différentes pour traduire des croquis en maillage 2D: la méthode des aires et la méthode des cycles .

B. Programme :

a. Main.py :

C'est le fichier principal du programme et celui qui doit être lancé.

En lançant main.py, une interface graphique permet à l'utilisateur de choisir la méthode qu'il souhaite utiliser, de renseigner les paramètres demandés par cette méthode, puis de générer le maillage souhaité.

L'interface est implémentée avec Tkinter.

La prévisualisation du maillage généré est implémentée avec Matplotlib, via pyplot.

b. Méthode aire :

i. Point de vue global :

Le principe de la méthode par aire est de commencer par l'identification des différentes aires du dessin et d'en déduire les segments nécessaires pour les représenter.

Cette méthode a pour avantage de directement gommer les erreurs humaines de tracé grâce à l'identification des meilleures séparations possibles entre les zones.

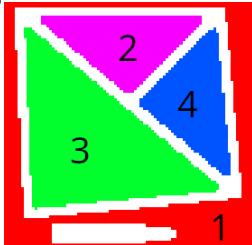
La méthode se divise en 6 étapes :

1. On transforme l'image (au format PNG par exemple) en un format plus exploitable :

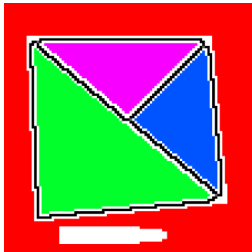
un tableau de booléens valant 1 si le pixel fait partie du tracé et 0 sinon	
image originale	image binaire



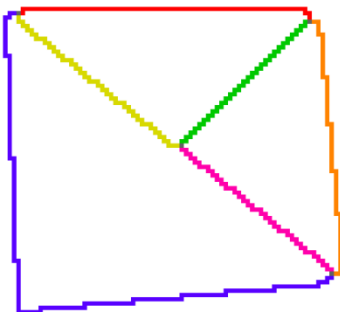
2. On identifie les différentes zones



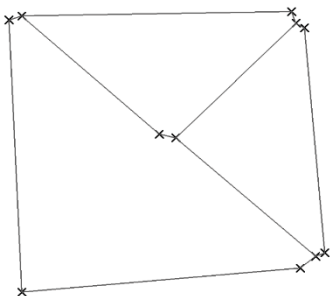
3. À partir des zones, on détermine les meilleurs limites entre elles



4. On groupe les points limites en “séparations”, une séparation est une ligne continue qui relie 2 croisements (points critiques) entre eux ou 1 point avec lui-même.



5. On simplifie chaque séparation en segments interprétables par gmsh.



6. On renvoie le mesh final dans un fichier .geo interprétable pas gmsh.

Les fichiers concernant la méthode aire sont :

main.py, utilitaires_main.py, Geometrie.py, Transcription_Image.py,
Determination_zones.py, get_separation.py, traces.py, outilsGeometrie.py, simplification.py
et gmsh_export.py .

ii. Geometrie.py :

Contient **Point()**, la classe utilisée pour modéliser un point par toutes les fonctions de la méthode aire.

iii. utilitaires_main.py :

Contiens des fonctions utilisées par main.py pour l'exécution de la méthode aire

iv. simplifier_separations(seps, epsilon) :

simplifie chaque séparation de la **liste de séparation** seps avec l'algorithme de **douglas peucker** de paramètres **epsilon**. (cf simplification.py)

v. nb_points_separations(seps) :

renvoie le nombre de points distincts utilisés par les séparations de la liste de séparation seps
,
l'obtient en ajoutant tous les points à un set puis mesurant la taille du set final.

vi. affiche_scatter(im, val) :

im : une grille d'éléments

val : un élément

écrit tous les points de im valant val sur la fenêtre de matplotlib.pyplot selon leurs coordonnées avec l'aide de plt.scatter

vii. get_separation.py :

Ce fichier est dédié à la résolution des étapes 3 et 4 de la méthode aire.

viii. TasMin() :

classe modélisant un tas min .

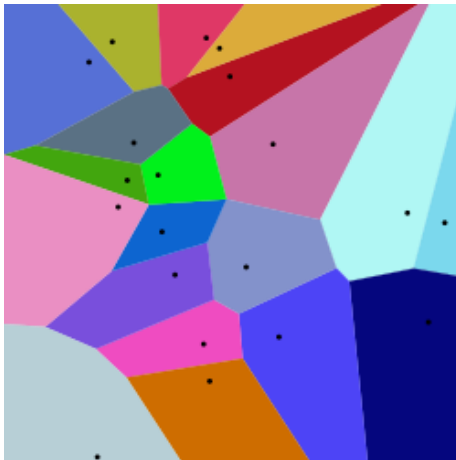
est utilisé par **get_separation(e_zones)** comme une file de priorité .

ix. get_separation(e_zones) :

fonction réalisant les étapes 3 et 4 de la méthode aire

l'**étape 3** consiste à **déterminer** les “meilleurs limites”, noté **séparations**, entre les zones identifiées.

La définition des séparations est inspirée des diagrammes de Voronoï :



Un diagramme de Voronoï en la division d'un plan à partir d'un ensemble de point initiaux en les « zones d'influence » de chaque point initial.

L'aire d'influence d'un point initial est l'ensemble des points du plan plus proches de ce point initial que de tous les autres points initiaux.

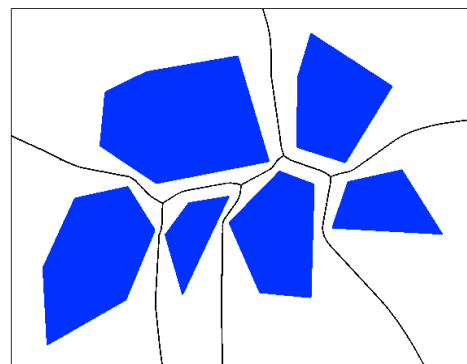
Pour trouver les séparations, nous considérons les « aires d'influence » de chacune des zones détecté

les zones sont représentées par des polygones bleus :

Ainsi les points faisant partie d'une séparation sont les points inclus dans les « aires d'influence » d'au moins 2 zones différentes.

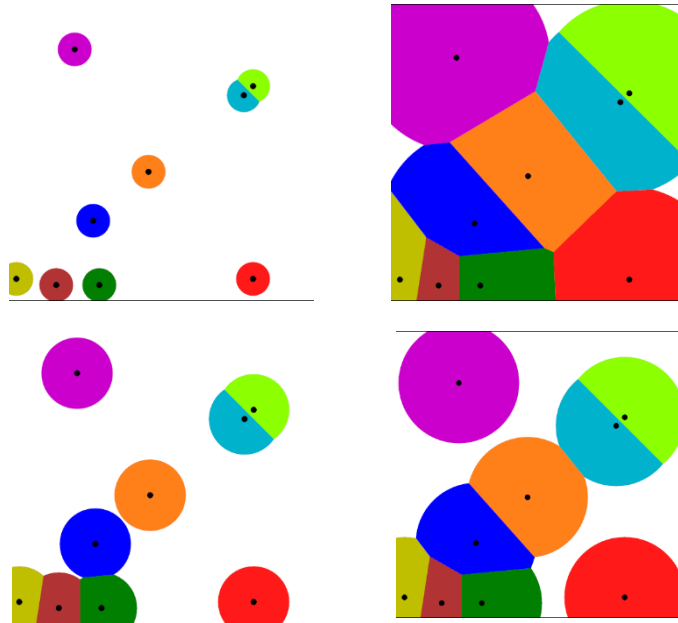
Ces points sont ici représentés en noir.

get_separation(e_zones) détermine les points des séparations en propageant les zones identifiées.



Complexité temporelle : $O(n \cdot \log(n))$ avec n le nombre de points de l'image

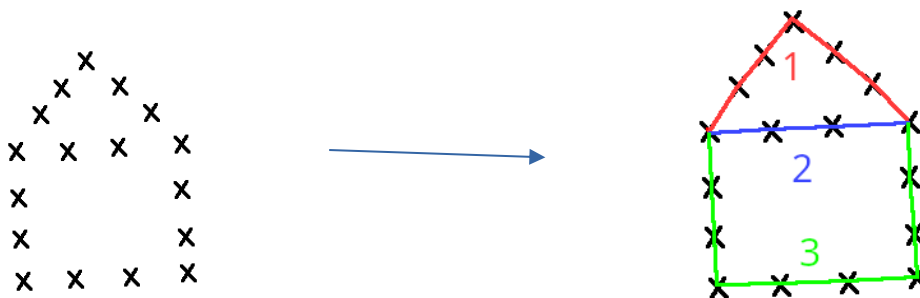
Ci-dessous un exemple de détermination des cellules de Voronoï par propagation des points initiaux :



Une fois que l'on a la liste des points constituant les séparations, on passe à **l'étape 4 : l'identification des séparations**.

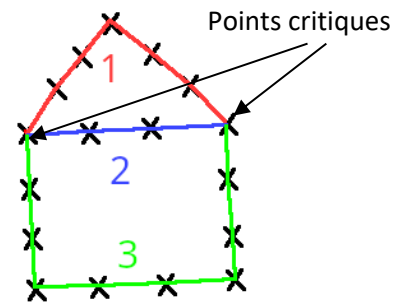
Tout ce que l'on a pour l'instant c'est une liste de point, hors pour avoir un maillage il faut une liste de points et de **liaisons** entre eux.

Donc dans la seconde partie de `get_separation(e_zones)` les points identifiés sont rassemblés en différentes séparations et ordonnée afin de construire le maillage et de permettre sa simplification.

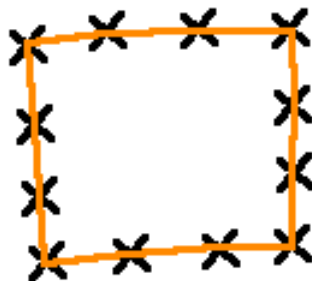


Le programme y arrive en parcourant les points identifiés de proches en proches d'un point critiques à un autre :

Les points critiques sont les points présents dans plusieurs séparations, ils déterminent leurs limites.



Les limites n'ayant pas de point critique sont celles formant des boucles et sont traitées séparément mais toujours déterminé par une exploration de proche en proche.



Complexité temporelle : $O(n)$

Ainsi `get_separation(e_zones)` prend la représentation des zones et renvoie la liste des séparations avec une complexité temporelle $O(n \cdot \log(n))$ avec n le nombre de case de `e_zones`.

x. La méthode `transcription_image(fichier_image, seuil_gris)` :

Transcrit une image sous la forme d'un tableau (ie liste de listes, dans le code) de booléen ; tel que `True` désigne un pixel clair, dit blanc, et `False` un pixel foncé, dit noir, par rapport à une valeur de seuil, `seuil_gris`. Cette transcription s'effectue à partir d'un fichier image, `fichier_image`, au format png ; et de la valeur entière (de type int) de seuil, `seuil_gris`, entrés en arguments de la fonction.

Pseudo-code :

```
pour chaque pixel (ie chaque case ou élément du tableau) :
  on convertit le code rgb, en une valeur de nuance de gris ;
  on détermine l'état du pixel, blanc ou noir ;
  on insère le booléen correspondant dans le tableau qui sera renvoyé en sortie.
```

xi. La méthode `zones_depuis_image(image, seuil)` :

Détecte les zones blanches, délimitées par les tracés noirs. Cette fonction prend deux arguments en entrée : un tableau de booléen, transcrivant l'image sous la forme de la sortie de la fonction `transcription_image`, et une valeur de seuil, définissant la taille minimale acceptée d'une zone blanche, en nombre de cases. Cette fonction renvoie alors un tableau d'entiers, au sein duquel les cases noires sont codées par 0, et les cases blanches par un entier strictement positif. Au sein d'une même zone blanche, les cases sont codées par le même entier. La zone blanche externe, collée au bord de l'image prend la valeur 1 ; et les suivantes prennent les entiers immédiatement supérieurs. On remarque que la valeur des zones internes dépend de l'ordre de parcours et que la valeur de la dernière zone détectée correspond au nombre total de zones.

La détection d'une zone blanche est effectuée grâce à un parcours en largeur.

Principe de la fonction :

pour chaque case :

si elle est noire :

on l'indique ;

sinon, si elle est blanche et non encore visitée :

on effectue le parcours de la nouvelle zone blanche, en indiquant sa valeur

dans chacune de ses cases.

NB : La fonction parcourt toutes les cases de l'image dans l'ordre suivant : d'abord la frontière, puis l'intérieur ligne par ligne ; de façon à faciliter l'incrémementation de la valeur des zones

Objectifs :

Une séparation entre deux zones est définie par plusieurs éléments caractéristiques :

- les identifiants des deux zones qu'elle délimite ;
- les identifiants des deux points critiques correspondant au début et à la fin de la séparation (ces points pouvant éventuellement être confondus) ;
- l'ensemble des points limites appartenant à ces deux zones et constituant la séparation.

L'objectif du travail qui m'a été confié est de permettre la représentation de chaque séparation de zones à l'aide de primitives géométriques adaptées, à savoir **des lignes brisées, des arcs de cercle ou des courbes**.

Pour atteindre cet objectif, il a été nécessaire de :

- **définir un format d'enregistrement** des tracés capable de représenter ces différents types de géométries ;

- concevoir une méthode permettant, à partir d'un ensemble de points décrivant une séparation, **d'identifier automatiquement le type de tracé le plus approprié et de l'associer à cette séparation.**

Architecture Code :

Le travail a été structuré en plusieurs modules Python distincts, chacun ayant une responsabilité clairement définie. Le code est ainsi réparti en quatre fichiers principaux : **traces.py** , **fitting.py**, **simplification.py** et **outilsGeometrie.py**.

Méthode adoptée :

- Extraction des séparations sous forme de points bruts issus des pixels de l'image.
- Simplification des tracés à l'aide de l'algorithme de Douglas–Peucker afin de réduire le nombre de points tout en conservant la forme globale.
- Ajustement de différents modèles géométriques candidats (ligne droite, arc de cercle, courbe).
- Calcul de l'erreur d'approximation pour chaque modèle géométrique testé.
- Sélection du tracé présentant la meilleure adéquation avec les points de la séparation.
- Utilisation du tracé retenu pour une représentation compatible avec le logiciel Gmsh.

xii. Traces.py :

Ce module contient la définition des structures de données représentant **les différents types de tracés géométriques** utilisés **pour modéliser les séparations entre zones**.

Il définit une classe abstraite Trace, décrivant les informations communes à toute séparation (zones associées, points de début et de fin), ainsi que des classes spécialisées :

- Ligne, pour représenter une séparation rectiligne ;
- ArcCercle, pour représenter une séparation sous forme d'arc de cercle ;
- Courbe, pour représenter une séparation plus générale à l'aide d'une courbe (spline).

Chaque classe implémente une méthode de calcul d'erreur permettant d'évaluer la qualité de l'approximation du tracé par rapport aux points issus de l'image, ainsi qu'une méthode d'export dans un format interprétable par le logiciel Gmsh.

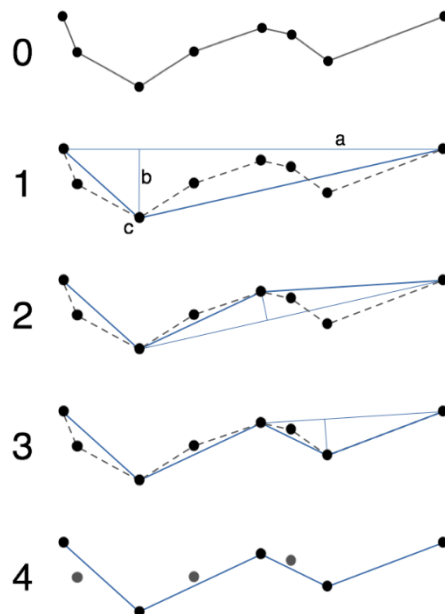
xiii. Simplification.py :

Ce module est dédié au prétraitement des données issues de l'image.

Il implémente **l'algorithme de Douglas–Peucker**, utilisé pour simplifier les tracés en réduisant le nombre de points tout en conservant la forme géométrique globale de la séparation. Cette étape permet de limiter le bruit, d'améliorer la stabilité des traitements ultérieurs et de réduire la complexité des calculs.

La simplification constitue une étape indépendante, réalisée avant la reconnaissance du type de tracé.

Comment fonctionne l'algorithme de Douglas Peucker ?



L'algorithme travaille de manière récursive par la méthode « diviser pour régner ». À l'initialisation on sélectionne le premier et le dernier point (cas d'une polyligne), ou un point quelconque (cas d'un polygone). Ce sont les bornes initiales. À chaque étape on parcourt tous les points entre les bornes et on sélectionne le point le plus éloigné du segment formé par les bornes :

1. s'il n'y a aucun point entre les bornes l'algorithme se termine,
2. si cette distance est inférieure à un certain seuil on supprime tous les points entre les bornes,
3. Si elle est supérieure, la polyligne n'est pas directement simplifiable. On appelle de manière récursive l'algorithme sur deux sous-parties de la polyligne : de la première borne au point distant, et du point distant à la borne finale.

xiv. Fitting.py :

Ce module assure **la reconnaissance et la sélection du tracé géométrique le plus adapté à une séparation donnée.**

À partir d'un ensemble de points simplifiés, il teste **différents modèles géométriques candidats** (ligne, arc de cercle, courbe) en **évaluant leur erreur d'approximation**. Le tracé présentant la meilleure adéquation avec les points est alors sélectionné et associé à la séparation.

Ce module joue ainsi un rôle central dans la décision du type de représentation géométrique à utiliser pour chaque séparation.

xv. OutilsGeometrie.py :

Ce module regroupe **l'ensemble des fonctions géométriques élémentaires utilisées par les autres parties du programme.**

Il contient notamment des fonctions de calcul de distances, telles que **la distance entre un point et un segment**, nécessaires à la fois pour l'algorithme de simplification et pour l'évaluation de l'erreur des différents modèles géométriques. Centraliser ces fonctions dans un module dédié permet d'éviter la duplication de code et d'assurer une cohérence des calculs géométriques.

c. Méthode Cycles

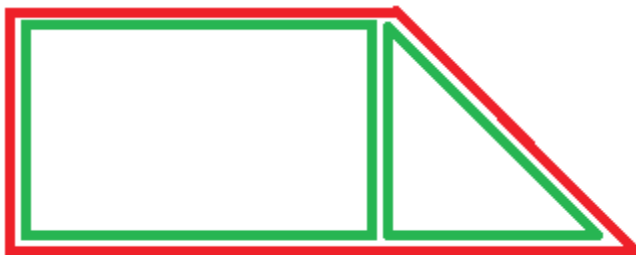
i.filterCycles()

Rôle :

Filtre les cycles élémentaires des cycles superflus.

Explication :

Puisque le parcours du graphe de l'image détecte tous les cycles possibles, il est nécessaire de supprimer les redondances :



Ici, les cycles verts sont «élémentaires» alors que le cycle rouge est superflu.

Fonctionnement :

Tant qu'il existe au sein de la liste des cycles un cycle d'indice $i3$ se révélant être la somme de deux cycles d'indices $i1$ et $i2$ par le test «assert not Somme(...)», on supprime le cycle $i3$ de la liste.

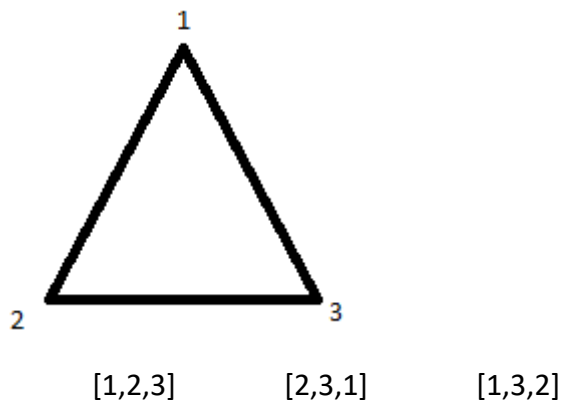
ii.same_cyc(cyc1,cyc2)

Rôle :

Détermine si *cyc1* et *cyc2* décrivent un même cycle.

Explication :

Au fil du parcours du graphe à la recherche des cycles, il est possible qu'on retrouve un cycle déjà découvert, soit parce que le premier sommet du cycle est différent, soit parce que le cycle est exploré dans l'autre sens.



Ces trois tableaux décrivent un même cycle, le second en commençant par le sommet n°2, le troisième en tournant en sens horaire.

Fonctionnement :

Après avoir vérifié que *cyc1* et *cyc2* ont la même longueur, on cherche l'indice *i0* du premier sommet de *cyc1* dans *cyc2*, puis on vérifie si *cyc2* revient à *cyc1* avec un décalage d'indice et/ou en parcourant les sommets à l'envers.

iii. *new_cyc()*

Rôle :

Détecte de nouveaux cycles pendant le parcours du graphe, à chaque nouveau sommet exploré.

Fonctionnement :

En s'appuyant sur la matrice d'adjacence du graphe, on vérifie si le nouveau sommet permet de former des cycles à partir de la pile des sommets explorés et que ces cycles n'ont pas encore été décrits dans *cyc_lst*, auquel cas ces cycles y sont ajoutés.

iv. *explore()*

Rôle :

Explore le sommet suivant pendant le parcours du graphe

Fonctionnement :

On étudie le sommet *current* se trouvant au sommet de la pile, puis on ajoute successivement ses voisins n'ayant pas encore été explorés. Pour chacun d'entre eux, on effectue le test *new_cyc* avant de les étudier à leur tour. Une fois que tous ces voisins ont été explorés, on retire *current* de la pile.

v. isIn(extCyc,intCyc)

Rôle :

Détermine si *intCyc* est à l'intérieur de *extCyc*.

Explication :

Lors de la modélisation des surfaces, il se peut qu'un cycle soit positionné à l'intérieur d'un autre, auquel cas il faudra retirer la surface du cycle intérieur à celle du cycle extérieur.

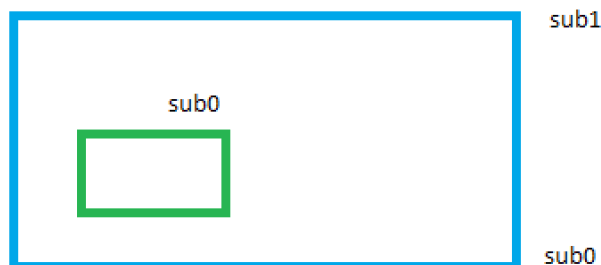


Ici, le cycle vert étant à l'intérieur du cycle bleu,

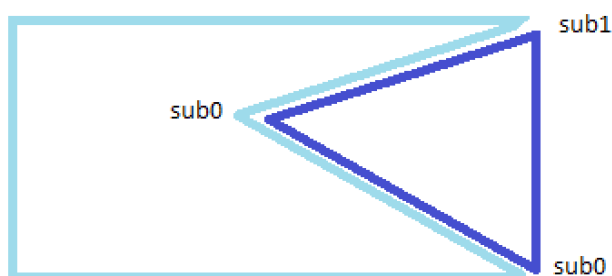
La surface bleu clair à décrire présente un trou.

Fonctionnement :

Après avoir vérifié que les cycles sont bien disjoints, on sélectionne trois sommets quelconques : *sub0* pris dans *intCyc*, *sub1* et *sub2* pris dans *extCyc*.



Le test s'appuie sur le principe suivant :



Puisque *sub0* est positionné dans *extCyc*, la somme des cycles bleu clair et bleu foncé donne *extCyc*.

Ainsi, le test de somme entre le cycle «triangulaire» et le cycle «réduit du triangle» suffit à détecter l'inclusion de *intCyc* dans *extCyc*.

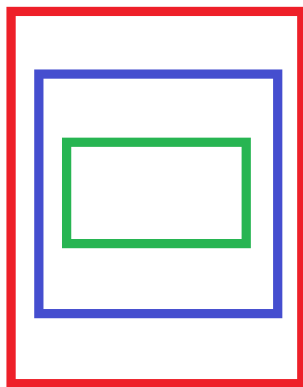
vi. findSurfaces()

Rôle :

Modélise les surfaces du graphe.

Fonctionnement :

On initialise toutes les surfaces par les surfaces simples des cycles élémentaires, auxquelles on retirera les éventuelles inclusions. Afin de gérer les cas où plusieurs inclusions successives existent, on procède ainsi :



cyc_lst = [cycle rouge, cycle bleu, cycle vert]

On génère la matrice d'inclusion correspondante, dont le coefficient *i,j* indique si le cycle *i* est inclus dans le cycle *j* :

	Faux	Faux	Faux
<i>incl_mat</i> =	Vrai	Faux	Faux
	Vrai	Vrai	Faux

L'objectif est de détecter le cycle le «plus extérieur», ici le cycle rouge, et de lui retirer l'intérieur des seconds cycles les «plus extérieurs», ici seulement le cycle bleu. Une fois cela fait, le dictionnaire *done* permettra de négliger le cycle le «plus extérieur» afin de réitérer le processus sur les cycles restants. Cette analyse s'arrête lorsque aucun cycle ne peut plus être

considéré, auquel cas toutes les surfaces auront été établies correctement. La détection du «rang d'extériorité» se fait selon *incl_count_lst*, une liste dont l'élément *i* indique le nombre de cycles non-négligés dans lesquels le cycle *i* est inclus. Lors d'une itération du processus, les seconds cycles les «plus extérieurs» sont ceux contenus une seule fois par un cycle non-négligé. Une fois ceux-ci déterminés, leur intérieur est retiré de la surface du cycle concerné, lequel devient négligé pour l'itération suivante.

Somme de Figures

Objectifs :

Le programme réalisé a pour objectif de reconnaître si la combinaison de deux figures géométriques permet de reconstruire une troisième figure.

Autrement dit, le programme cherche à vérifier une relation du type :

$$c1 + c2 = c3$$

où *c1*, *c2* et *c3* représentent chacune une figure.

Chaque figure est décrite par une liste de sommets représentant le contour de la forme.

Exemple :

['a', 'b', 'd']

correspond à une figure composée des sommets a, b et d.

Le programme détermine alors si l'union des deux premières figures permet bien d'obtenir exactement la troisième figure.

Principe général :

La première construit les arêtes des figures à l'aide d'une liste de sommets, donnée de telle sorte que deux sommets consécutifs dans la liste sont reliés sur la figure. La deuxième fonction prend comme variables deux listes de sommets. Son but est de déterminer les arêtes communes entre les deux figures données par les listes. Quant à la dernière fonction, elle prend en paramètres trois listes de sommets. Son but est de déterminer si la fusion des deux premières figures *c1* et *c2*, forment la troisième figure *c3*.

Le programme :

1. construit les arêtes de chaque figure ;
2. recherche l'arête commune entre les deux premières figures ;
3. parcourt le contour extérieur de l'union des deux figures ;
4. reconstruit la figure obtenue ;
5. comparez cette figure avec la figure attendue.

Si les contours correspondent exactement, alors le programme retourne True.

Sinon, il retourne False.

vii.

construire_aretes(sommets) :

Cette fonction permet de transformer une figure décrite par une liste de sommets en un ensemble d'arêtes.

Une arête correspond au segment reliant deux sommets consécutifs.

Explication :

- len(sommets) récupère le nombre de sommets.
- set() permet d'éviter les doublons.
- Le modulo %n reconnecte automatiquement le dernier sommet au premier.
- frozenset permet de considérer que $AB = BA$.

Exemple :

['a', 'b', 'd']

donne les arêtes :

ab, bd, da

viii. aretes_communes(liste1,liste2) :

Cette fonction permet de trouver les arêtes partagées par deux figures.

Explication :

L'opérateur « & » représente l'intersection entre deux ensembles.

Le résultat contient uniquement les arêtes présentes dans les deux figures.

ix. Somme(c1,c2,c3) :

La fonction permet de vérifier si l'union de deux figures géométriques c1 et c2 permet de reconstruire une troisième figure c3.

Principe de fonctionnement :

Le programme commence par rechercher une arête commune entre les deux figures.

Si les figures ne possèdent pas d'arête commune, alors la somme est considérée comme invalide et le programme retourne False.

Les deux sommets composant cette arête commune sont ensuite récupérés afin de servir de points de transition entre les deux figures.

Le programme choisit ensuite un sommet de départ dans la première figure qui ne fait pas partie de l'arête commune. À partir de ce sommet, il reconstruit progressivement le contour extérieur de la figure obtenue.

Pour cela, l'algorithme parcourt les sommets de la première figure. Lorsqu'il atteint l'un des sommets de l'arête commune, il bascule automatiquement dans la deuxième figure.

Le programme choisit alors le bon sens de parcours dans cette deuxième figure afin d'éviter de repasser immédiatement sur l'arête commune. Cette étape permet de suivre correctement le contour extérieur de la figure finale.

Tous les sommets parcourus sont stockés dans une liste appelée union.

Une fois le parcours terminé, le programme compare les arêtes de la figure reconstruite avec celles de la figure attendue c3.

Si les deux ensembles d'arêtes sont identiques, alors le programme retourne True, il retourne donc False dans le cas contraire.

Détection et Extraction de Contours

Le module développé a pour objectif de transformer une image en un ensemble structuré de points exploitables. Concrètement, il doit accomplir les tâches suivantes :

- Charger une image depuis le système de fichiers
- Effectuer un prétraitement pour améliorer la qualité des données
- Détecter tous les contours présents, qu'ils soient externes ou internes
- Extraire les coordonnées (x, y) des points de chaque contour détecté
- Fournir une visualisation permettant de valider les résultats obtenus

Le traitement est structuré en cinq étapes successives et indépendantes. Chaque étape est encapsulée dans une fonction dédiée, ce qui garantit une bonne modularité, une meilleure lisibilité du code et une maintenance facilitée.

1. Prétraitement de l'image
2. Binarisation
3. Détection des contours
4. Extraction des points
5. Visualisation des résultats

x. Prétraitement de l'image - preprocess(img)

Le prétraitement est une étape fondamentale pour garantir la qualité de la détection des contours. L'image est d'abord convertie en niveaux de gris afin de simplifier l'analyse. Ensuite, un flou gaussien est appliqué pour réduire le bruit et éviter la détection de faux contours. Enfin, une binarisation automatique (méthode d'Otsu) sépare les objets du fond, et un nettoyage morphologique finalise cette étape.

En résumé, les actions réalisées sont les suivantes :

- Conversion en niveaux de gris
- Flou gaussien pour atténuer le bruit
- Binarisation automatique par seuillage d'Otsu
- Nettoyage morphologique (ouverture)

Objectif : obtenir une image propre où les contours sont nets et facilement détectables.

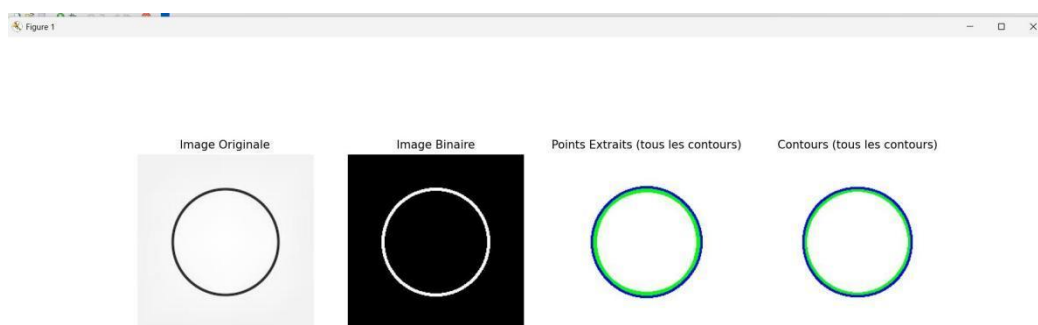
xi. Détection des contours - `extract_points(binary)`

La détection des contours est réalisée à l'aide de la fonction

La détection utilise `cv2.findContours` de la bibliothèque OpenCV, avec deux paramètres clés :

- `RETR_TREE` : récupère l'ensemble des contours, y compris les contours internes imbriqués
- `CHAIN_APPROX_NONE` : conserve la totalité des points de chaque contour, sans simplification

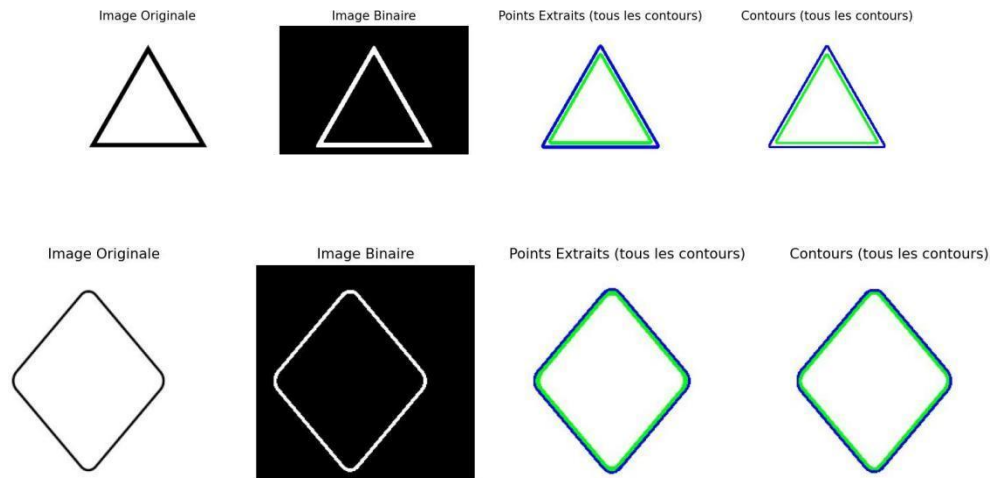
Ce choix est particulièrement important pour les formes complexes contenant des structures imbriquées — par exemple, un carré contenant d'autres carrés. L'objectif est de ne rien perdre, même les lignes internes ou les détails les plus fins.



xii. `draw_points(img_shape, all_contours_points)`

Cette fonction se charge d'afficher visuellement les points extraits. Pour rendre la lecture

aisée, chaque contour est représenté avec une couleur distincte, et les points sont dessinés individuellement. Cela permet de vérifier rapidement la qualité de l'extraction avant toute utilisation ultérieure.

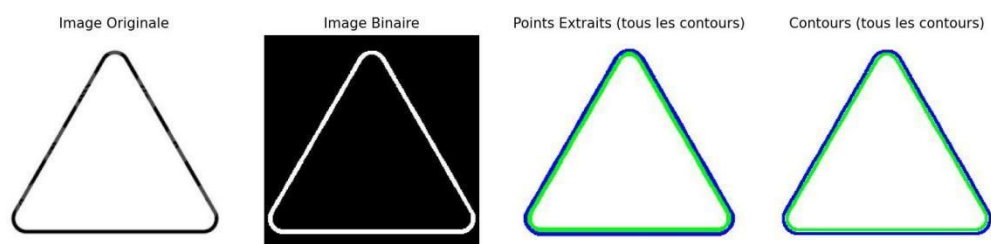


xiii. Reconstruction des contours - `draw_contours(img_shape, all_contours_points)`

Cette fonction reconstruit les contours exacts à partir des points extraits, sans introduire de déformation. La décision de ne pas utiliser de splines est délibérée, pour plusieurs raisons :

- Les splines lissent les formes polygonales et arrondissent les angles
- Elles créent des artefacts indésirables aux intersections
- Elles ne conviennent pas aux formes à vocation technique ou industrielle

La solution retenue utilise `cv2.polylines` pour garantir une représentation fidèle de la géométrie, sans aucune déformation.



xiv. Visualisation des résultats

Une étape de visualisation est intégrée au pipeline pour valider les résultats à chaque exécution. Trois images sont affichées en parallèle :

- L'image originale
- L'image binarisée après prétraitement
- L'image reconstruite à partir des contours détectés

